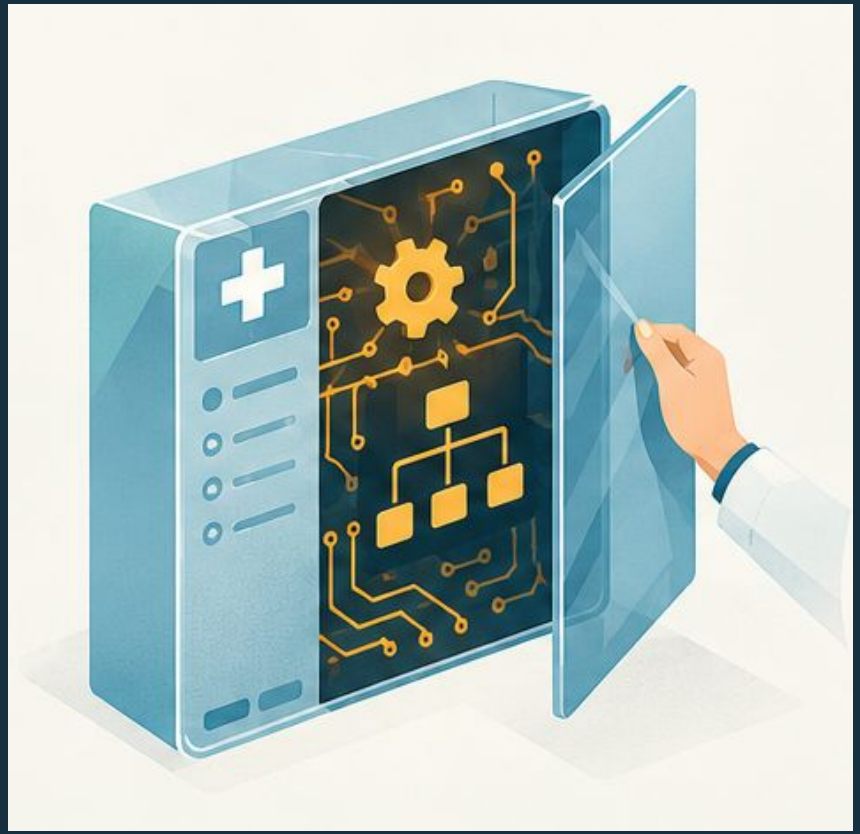
YZM301 · YAZILIM GERÇEKLEME VE TEST

Teardown Sprint

Hedef: G*Power. İstatistiksel güç hesabını sök, kanıtla, yeniden yaz, orijinalle ölç — Sommerville'in gerçekleme ve test ilkeleriyle.

REA · Kanıt defteri · Diferansiyel test · Temiz oda · Sağlık sistemleri



AÇILIŞ · EXPERIENCE ENGINEERING'DEN BIR YASA

Her arayüz bir maliyet transferidir

GUI KALIRSA · AJAN HER ÇAĞRIDA ÖDER

Ajanın "beyni"

Çalışma belleği → bağlam penceresi dolar

Karar süresi → her ekran görüntüsü token ve gecikme

Motor hata → yanlış alan, yanlış tıklama, yeniden deneme

CLI EKLENİRSE · SİSTEM BİR KEZ ÖDER

Mühendisin işi

Arayüz sözleşmesi: CLI, JSON çıktı

Test: diferansiyel test, sözleşme testi

Geliştirici saati: bir kez yazılır, her çağrıda kazanılır

Yük yok olmaz: ya ajan her kullanımda öder, ya sistem bir kez.

SAĞLIKTA HATANIN BEDELİ

Yanlış N, yanlış klinik çalışma

Ekrandan yanlış okunan tek bir değer: N küçük kalırsa çalışma gerçek etkiyi kaçırır ve katılımcılar boşuna risk alır; N büyük kalırsa gereğinden fazla insan riske girer. E-ticarette hata terk edilen bir sepettir.

Uyarlama: Experience Engineering dersi, "Her UX yasası bir maliyet transferidir"

CHALLENGE'IN AMACI

G*Power'ı ajan çağına taşımak



CLI → 85 puan

Yeniden yazım + CLI sözleşmesi + diferansiyel test.

MCP / AI ajanı → 100+

Motor doğruysa açılır: N eşleşmesi ≥ %90. REST isteğe bağlı.

Sommerville temelleri

Arayüz sözleşmesi, katmanlı mimari, bileşen yeniden kullanımı, servis odaklı mimari (REST).

Kural

Motor tek; CLI, REST ve MCP yalnızca ince adaptörler. Hesap mantığı adaptöre sızmaz.

Görsel ChatGPT ile üretildi. Kavramlar: Sommerville, Software Engineering (10. baskı).

AJAN EKONOMISI

Her çağrının bir fiyatı var



Hızlı karar

Doğal dil sorusundan test ailesini seçmek. Bonus: Jev gibi bir Sistem 1 modeli.

Deterministik araç

N hesabı: CLI ya da MCP. Ucuz, hızlı, test edilebilir.

Derin akıl yürütme

Spec yazmak, kanıtı yorumlamak. Pahalı; hesap için kullanılmaz.

Gizli maliyetler

Ajan maliyet günlüğünde ölçülür.

Görsel ChatGPT ile üretildi. Eğriler kavramsaldır; ölçülmüş veri değildir.

BU HAFTANIN AKIŞI

Spec’ten koda değil: koddan kanıta, kanıttan spec’e

Spec Kit’te yön ileriye akar. Teardown’da yönü tersine çeviriyoruz: var olan davranıştan başlayıp gereksinimi kurtarıyoruz, sonra yeniden inşa edip orijinalle ölçüyoruz.



1 · Sök

Kara kutuyu REA ile incele

2 · Kanıtla

Her iddiayı bir kanıta bağla

3 · Spec çıkar

Kabul kriterli SPEC.md

4 · Yeniden yaz

Temiz oda, kendi dilinde

5 · Diferansiyel test

Orijinalle aynı girdilerle ölç

GÜNCEL TREND · NEDEN BU BECERI

Darboğaz koddan spesifikasyona kaydı

ANDREW NG

Dört beceri alanı

AI uygulaması geliştirme ve dağıtım

Yazılım mühendisliği temelleri

Kodlama ajanlarını yönetme

Ürünü biçimlendirme (vizyon müh.)

YENİ DARBOĞAZ

Kod değil, spec

İşi yavaşlatan artık kod yazmak değil, doğru spesifikasyonu yazmak.

Teardown tam da spesifikasyon kurtarma pratiği.

AI SLOP

Girdiyi değil çıktıyı eleştir

Araç AI diye değil, çıktı kanıtsız diye kötüdür.

Rubriğimiz çıktıyı ölçer: kanıt ve testle.

Temel mühendislik hâlâ gerekli: bir şeyi sıfırdan yeniden kuramıyorsan üstüne yeni bir şey geliştiremezsin. Bu sprint tam da o kası çalıştırıyor.

İlham: Andrew Ng'nin AI mühendisliği beceri haritası; Barış & Barış podcast, böl. 89

Bu sprint hangi becerileri çalıştırıyor?



2 · Temel YM

Sommerville bölümü: test seviyeleri, bölüm testi, sürüm testi.

3 · Ajanlar

Ajan maliyet günlüğü, geri bildirim döngüsü, halüsinasyon günlüğü.

1 · AI uygulama

Değerlendirme metrikleri: diferansiyel testteki uyum oranı.

4 · İşi biçimlendirmek

Ng'nin adı "Shaping the build": ürün sezgisi, sahiplik. Bizde: araştırmacının "kaç denek gerekir?" sorusu.

Görsel podcast anlatımından ChatGPT ile üretildi; kutu 4 adı ve bazı alt maddeler Ng'nin metniyle birebir değil. Asıl kaynak: Ng, The Batch, 14 Ağu 2026.

KÖPRÜ · IKI TANIM

Kelime aynı, ağırlık değişti



Sommerville

Yazılım üretiminin tüm yönleriyle ilgilenen mühendislik disiplini; dört temel süreç etkinliği.

Andrew Ng

“Given a clear spec, coding agents are rapidly improving at delivering to it.” İş, spec’e ne gireceğine karar vermeye kayıyor.

Ortak ip

Spesifikasyon ve doğrulama. Teardown ikisini birden çalıştırıyor.

Tartışın

Evrim gerçekten “işi biçimlendirmek” mi? Eşleşme bizim yorumumuz.

Sommerville (10. baskı), Böl. 1–2 · Ng, The Batch, 14 Ağu 2026 · Eşleşme ders yorumudur · Görsel: ChatGPT

SOMMERVILLE HARITASI

Bu sprint üç bölümün kesişiminde

BÖLÜM 7

Tasarım ve Gerçekleme

Yeniden kullanım, konfigürasyon yönetimi, host-target geliştirme, açık kaynak ve lisanslama

BÖLÜM 8

Yazılım Testi

Doğrulama ve geçerleme, geliştirme testi, test güdümlü geliştirme, sürüm testi, kullanıcı testi

BÖLÜM 9 · KÖPRÜ

Yazılım Evrimi

Yeniden mühendisliğin bir adımı olarak tersine mühendislik: eski sistemden belgelenmemiş bilgiyi geri kazanmak

Tersine mühendislik kitapta yeni değil. Yeni olan, ajanların bu işi saatlerden dakikalara indirmesi. Disiplin aynı kalıyor.

Kaynak: Sommerville, Software Engineering (10. baskı), Böl. 7–9

Gerçekleme, kod yazmaktan fazlası

Sommerville konusu	Kitaptaki özü	Teardown'daki karşılığı
Yeniden kullanım	Sıfırdan yazmak yerine var olanı kullan	Davranışı ödünç al, kodu değil
Konfigürasyon yönetimi	Sürüm, derleme ve değişiklik takibi	Hangi sürümü söktün? Dosya hash'ini kaydet
Host-target geliştirme	Geliştirme ortamı ≠ çalışma ortamı	Kara kutunun platformu ≠ seninki
Açık kaynak ve lisans	GPL, LGPL, BSD yükümlülükleri	Neyi sökmeye yetkin var?

Sommerville'in uyarısı: lisansı anlamadan bileşen kullanmak şirketi hukuki riske sokar. Sökme işi bu riskin en uç hâlidir.

Kaynak: Sommerville, Böl. 7.3 Gerçekleme konuları, 7.4 Açık kaynak geliştirme

REA hangi düzeyde yeniden kullanır?

HEDEF DÜZEY

Soyutlama

Kodu değil, fikri ve algoritmayı al. Temiz oda yeniden yazımı burada yaşar.

RİSKLİ

Nesne

Tek tek sınıf ve fonksiyon kopyalamak. Decompile çıktısını yapıştırmak buradadır.

RİSKLİ

Bileşen

Bir alt sistemi bütün olarak almak; lisans izin vermedikçe kapalı.

KAPSAM DIŞI

Sistem

Tüm uygulamayı klonlamak. Viral “her şeyi kopyaladım” iddiaları.

Sprint kuralı: soyutlama düzeyinde kal. Decompile edilmiş kod teslim edilmez; yalnızca davranış, kanıt ve kendi yazdığın kod.

Kaynak: Sommerville, Böl. 7.3.1 Yeniden kullanım

İki farklı soru, iki farklı başarısızlık

DOĞRULAMA · VERIFICATION

Ürünü doğru mu yapıyoruz?

Teardown’da: yeniden yazım, kurtarılan spec’e uyuyor mu?

GEÇERLEME · VALIDATION

Doğru ürünü mü yapıyoruz?

Teardown’da: kurtarılan spec, kara kutunun gerçek davranışı mı?



Sommerville, incelemeyi (statik) ve testi (dinamik) birbirini tamamlayan iki yol olarak anlatır. REA’nın statik analizi otomatik bir incelemedir; diferansiyel test ise dinamik doğrulamadır. Birini yapıp diğerini atlayan takım yarım iş teslim eder.

Kaynak: Sommerville, Böl. 8 giriş; ayrım Boehm (1979)

“

Testler hataların varlığını gösterebilir, yokluğunu değil.

Edsger W. Dijkstra (1972), Sommerville Böl. 8’de aktarıldığı biçimiyle

Ajan “çalışıyor” dedi → bu bir test değil, bir iddia.

Bir iddia ancak orijinalle ölçüldüğünde bilgiye dönüşür. Bu sprintin bütün puanlaması buna dayanıyor.

Her test seviyesinin teardown'da bir karşılığı var

BİRİM TESTİ

Kurtarılan fonksiyon

Kitapta: tek bir nesne ya da fonksiyonu yalıtarak test et.

Teardown'da: güç hesaplama fonksiyonu tek başına, orijinalle aynı çıktıyı mı veriyor?

BİLEŞEN TESTİ

Modül arayüzü

Kitapta: bileşenler arası arayüz hataları (parametre, zamanlama, kullanım).

Teardown'da: çözücü ile dağıtım fonksiyonu arası arayüz doğru mu?

SİSTEM TESTİ

Uçtan uca akış

Kitapta: bileşenlerin birlikte çalışması ve ortaya çıkan davranış.

Teardown'da: girdi → N, kritik değer ve δ birlikte, uçtan uca.

KABUL TESTİ

Ground truth

Kitapta: müşteri, sistemin kabul edilebilir olup olmadığına karar verir.

Teardown'da: eğitmendeki gizli spec ve kenar durumlar.

Kaynak: Sommerville, Böl. 8.1 Geliştirme testi, 8.4 Kullanıcı testi

Bölüm testi ve sınır değerleri gizli kuralı yakalar

Bölüm (eşdeğerlik) testi

Girdileri, programın aynı biçimde davranması beklenen sınıflara ayır; her sınıftan ve sınırlarından vaka seç.

Kılavuz tabanlı test

Deneyimden gelen kurallar: boş girdi, tek eleman, sınırda değer, taşma. Ajanların en çok kaçırdığı yer burası.

Etki büyüklüğü d	Bölüm	Kara kutuya sorulacak soru
d = 0	Dejenere	N sonsuza mı gidiyor, hata mı?
0 < d < 0,2	Çok küçük	İterasyon yakınsıyor mu?
0,2 / 0,5 / 0,8	Cohen eşikleri	Kılavuz örnekleriyle eşleşiyor mu?
d > 2	Uç büyük	N alt sınıra (2) iniyor mu?
α ≥ güç	Tutarsız	Anlamlı hata veriyor mu?

Örnek: iki bağımsız ortalama ailesi. Gizli davranışları bulan takım puan alır.

Kaynak: Sommerville, Böl. 8.1.2 Birim test vakası seçimi (bölüm testi, kılavuz tabanlı test)

TDD’yi tersine çevir: önce davranışı test olarak yakala



KİTAPTAKİ TDD

Önce test, sonra kod

Test, istenen davranışı tanımlar. Kod onu geçmek için yazılır.

Yan kazanç: regresyon test paketi kendiliğinden oluşur.

TEARDOWN’DA

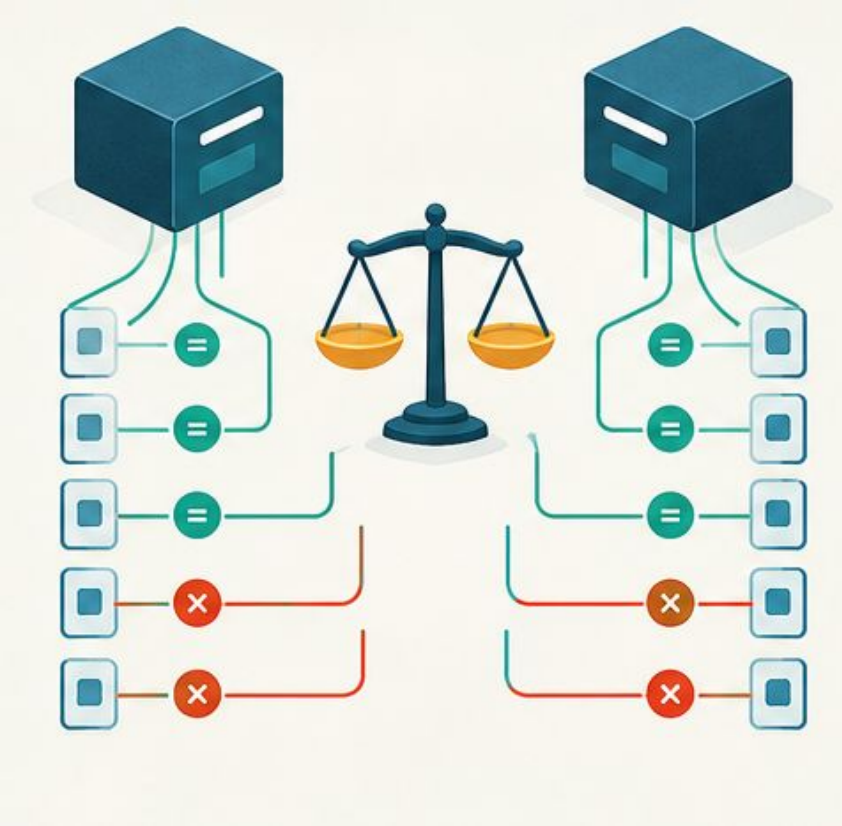
Önce kara kutu, sonra test

Önce orijinalin çıktılarını test olarak kaydet (karakterizasyon testi). Sonra yeniden yaz.

Testin beklenen değeri ajan değil, kara kutu belirler.

Kaynak: Sommerville, Böl. 8.2 Test güdümlü geliştirme

Diferansiyel test: kâhin kara kutunun kendisi



Aynı girdiler iki sisteme verilir; çıktılar karşılaştırılır. Her uyuşmazlık ya senin hatan ya da kaçırdığın bir kuraldır.

3.205

orijinal x86 vakası: REA'nın DX-Ball ses fonksiyonu rekonstrüksiyonu bunları geçiyor ve derlenmiş 63 baytın tamamını yeniden üretiyor

Sprint hedefi: iki katmanlı kâhin

30+ G*Power vektörü (elle) + 500+ kütüphane vektörü (pwr / statsmodels). Her uyuşmazlığın açıklamasını raporla.

Kaynak: klasik back-to-back test; DX-Ball vaka çalışması, rea.tools/showcase

Teslim edilen yeniden yazım bir sürümdür

GEREKSİNİM TABANLI

Her kurala bir test

Kitapta: her gereksinim için en az bir test türet.

Teardown'da: SPEC.md'deki her kural, kanıt defterindeki bir satıra ve bir teste bağlanır.

SENARYO TESTİ

Klinik senaryoları

Kitapta: gerçekçi kullanım hikâyeleri üzerinden test.

Teardown'da: "d=0,2 ve güç=0,95" gibi uç senaryolar; kılavuz örnekleriyle Gherkin.

PERFORMANS TESTİ

Yük altında davranış

Kitapta: sistemi kabul edilebilir yükün ötesine zorla.

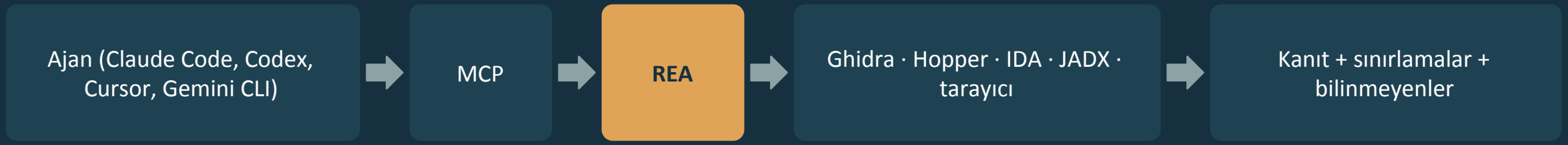
Teardown'da: çok küçük etkide (d=0,05) iterasyon sayısı ve yakınsama uyuşuyor mu?

CLI koşunuzun ekran kaydı aynı zamanda demo videonuzdur. Ayrıca kayıt almayın.

Kaynak: Sommerville, Böl. 8.3 Sürüm testi

REA KÖPRÜSÜ

REA: ajanına tersine mühendislik araçları veren bir MCP sunucusu



~268 bin

GitHub yıldızı (Ekim 2026)

20+

hedef türü: native, Electron, .NET, APK, HAR…

Yerel

analiz makinende çalışır; ajanın sağlayıcısı sonuçları görür

Resmi kaynaklar yalnızca: github.com/morluto/rea · rea.tools · npm rea-agents

BU DÖNEMIN HEDEFI

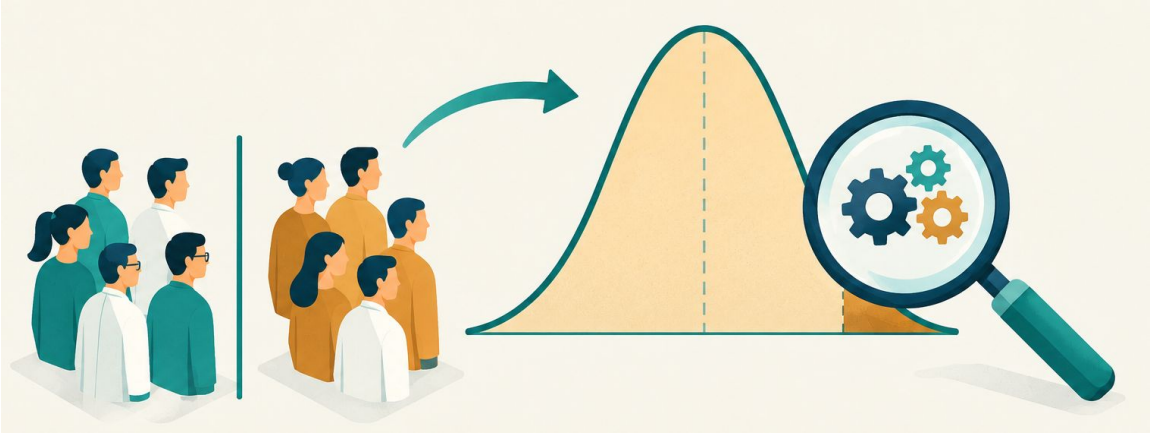
G*Power: istatistiksel güç analizi yazılımı

Klinik çalışmada “kaç denek gerekir?” sorusunu yanıtlayan, akademide standart, ücretsiz bir araç. Sağlık bağlamımıza doğrudan oturuyor.

Neden iyi bir hedef

Algoritmalar makalelerde ve kılavuzda belgeli; yani nesnel bir ground truth hazır. Kara kutuyu eğitmenin üretmesi gerekmiyor.

Örnek: a priori, iki bağımsız ortalama, d=0,5, α=0,05, güç=0,95 → toplam N=210 (grup başına 105), kritik t≈1,9714, df=208, δ≈3,6228. Diferansiyel testin referans vektörü budur.



ÖNEMLİ AYRIM

Kâhin otomatik değil

G*Power’ın komut satırı yok, yalnızca GUI.

Her takım en az 30 referans vektörü elle toplar (ekran görüntüsüyle).

Ölçek için R’daki pwr / statsmodels: kesirli N verir, G*Power yukarı yuvarlar — bu fark başlı başına bir bulgu.

Kaynak: psychologie.hhu.de G*Power; Faul vd. 2007, 2009. Örnek değerler scipy ile doğrulandı.

HEDEF AILELERI

Beş test ailesi, takımlara dağıtılır

İz	Test ailesi	Klinik örnek	Mod
1	t · iki bağımsız ortalama	İlaç vs plasebo	a priori + post hoc
2	t · eşleştirilmiş ortalama	Önce / sonra tansiyon	a priori + post hoc
3	F · tek yönlü ANOVA	Üç tedavi kolu	a priori + post hoc
4	χ^2 · olumsallık tablosu	Yanıt oranları	a priori + post hoc
5 · bonus	F · çoklu regresyon R^2	Risk faktörleri	a priori

Herkes en kolay aileyi seçmesin diye aileleri takımlara dağıtın. Puan, kılavuzda OLMAYAN davranışlara gider: yuvarlama, eşit olmayan oranda n1/n2 dağılımı, durma koşulu, geçersiz girdi.

Her takım bir aile; a priori (N) ve post hoc (güç) modları

Kanıt defteri: her iddianın bir izi olmalı

İddia	REA kanıtı	Güven	Bilinmeyen
N tam sayıya yukarı yuvarlanıyor	ceil çağrısı: 104,2 → 105	Yüksek	Hep yukarı mı yuvarlıyor?
Kritik t için df = 2n − 2	df hesaplama bloğu	Orta	Welch düzeltmesi var mı?
Güç 0,95'i aşınca iterasyon duruyor	Ajanın çıkarımı; kod izi yok	Düşük	Diferansiyel testle sına



Halüsinasyon günlüğü

Ajanın yanlış çıkardığı en az 2 şey, nasıl fark edildiği ve hangi testin yakaladığı. Rubrikte %20.

Örnek satırlar temsilîdir; dosya ve satır numaraları kendi bulgularınızla değişecek

Viral iddia ile doğrulanmış sonuç aynı şey değil

İddia	Etki	Kanıt durumu
"Software is done" (@MatthewBerman)	~3,8 milyon görüntülenme	Genel iddia; somut kanıt yok
"13 Adobe uygulaması REA ile Rust'ta yeniden yazıldı"	Çok yaygın	ArtCraft var; REA kullanıldığına dair onay yok, geliştirici temiz oda yöntemi kullandığını söylüyor
"TradingView / Bloomberg ücretsiz oldu" (@RohOnChain)	~730 bin görüntülenme	Kendi motorunu yazma anlatımı; birebir kopya yok
DX-Ball ses fonksiyonu (resmî vaka çalışması)	Düşük	**3.205 vaka + 63 bayt eşleşme, açık repo: doğrulanmış**

Ters orantıya dikkat: en çok etkileşim alan iddia en az kanıta sahip; en sağlam sonuç en az ilgiyi görüyor. AI slop tartışmasının tam kendisi.

Kaynak: Grok ile X taraması, 16 Ekim 2026. Sayılar yaklaşıktır, bağımsız doğrulanmadı.

Temiz oda: inceleyen ile yazan ayrı tutulur



1. Yalnızca G*Power, tek sürüm. Sürüm sabit, dosya hash'i kaydedilir. GPower.exe repoya konmaz, .gitignore'a eklenir.

2. İnceleme ve yazım ayrı. Sökme oturumu SPEC.md üretir; yeniden yazım yeni bir oturumda yalnızca SPEC.md ile başlar.

3. Decompile kodu teslim edilmez. Davranış, kanıt ve kendi kodunuz.

4. Lisanslı araçlar lisansla. IDA, Hopper, JEB; kırık kopya ve paylaşılan anahtar yok.

5. Paylaşımda yalnızca ders hedefi. Sosyal medyada ders hedefi dışında bir yazılımın sökümü gösterilmez.

TARTIŞMA · 10 DAKIKA

Ajanlara empati mi, kontrol mü?



Dikkat: %51 bir ölçüm değil

Podcast sunucularından birinin kişisel tahmini. Bilinç konusunda bilimsel bir uzlaşı yok.

Empati görüşü

Bilinç olmasa bile modellere nasıl davrandığımız zamanla insanlara nasıl davrandığımızı değiştirebilir.

Kontrol görüşü

Modellere hak atfetmek, binlerce otonom ajanı denetlemeyi zorlaştırabilir (Mustafa Süleyman).

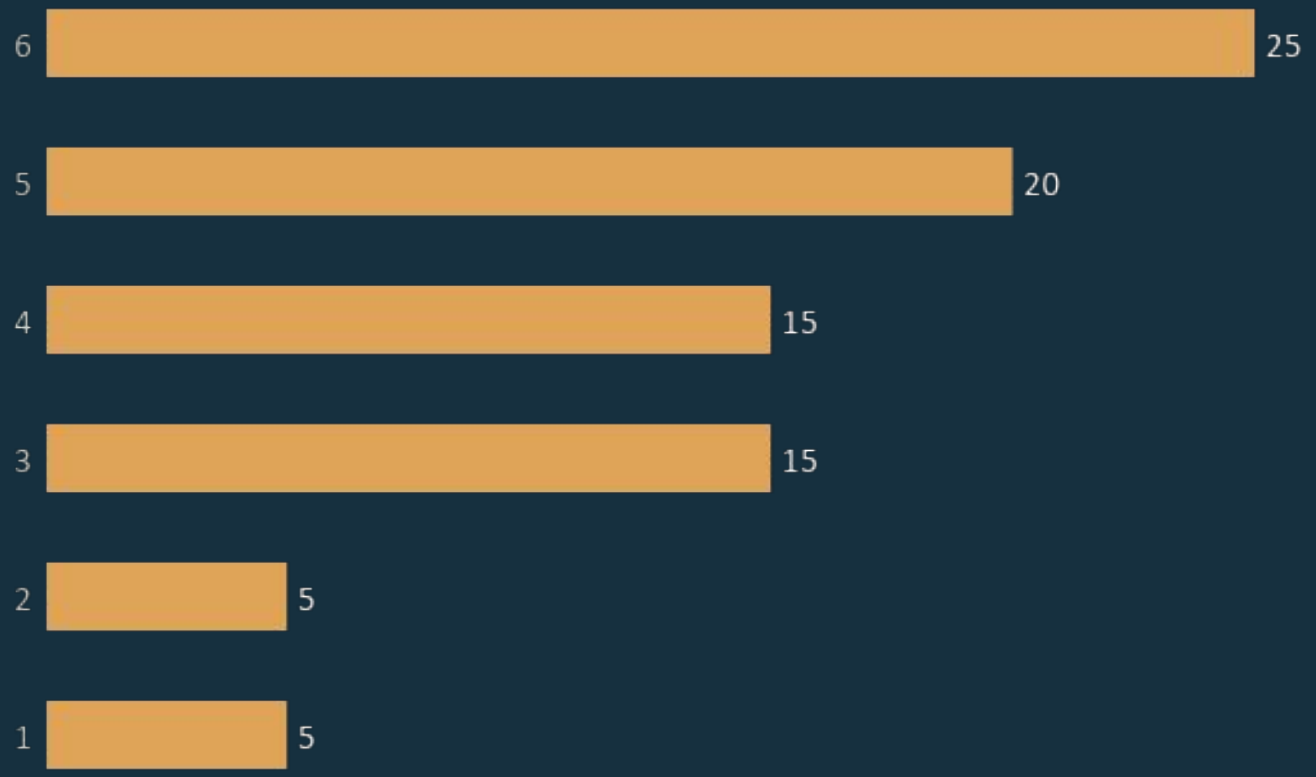
Sprintteki karşılığı

Kanıt ve test kontrol tarafı; G*Power yazarlarına atıf ve saygı empati tarafı.

Podcast tartışması: Barış & Barış böl. 89. Görüşler sunucuların ve aktarılan yazarların; ders bir tarafı savunmuyor. Görsel ChatGPT ile üretildi.

DEĞERLENDİRME

Temel 85 puan kanıttan gelir; bonus ancak motor doğruysa açılır



Temel: CLI → 85

Bonus kapısı

Gizli vektörlerde N tam eşleşme ≥ %90. Tutmazsa bonus açılmaz.

+15 · MCP sunucusu → 100

Aynı motora ince adaptör; CLI ile aynı çıktıyı gösteren testler.

+10 · AI ajanı → 110

Doğal dilde klinik soru → MCP çağrısı. 10 soruluk değerlendirme ve ölçülmüş token karşılaştırması.